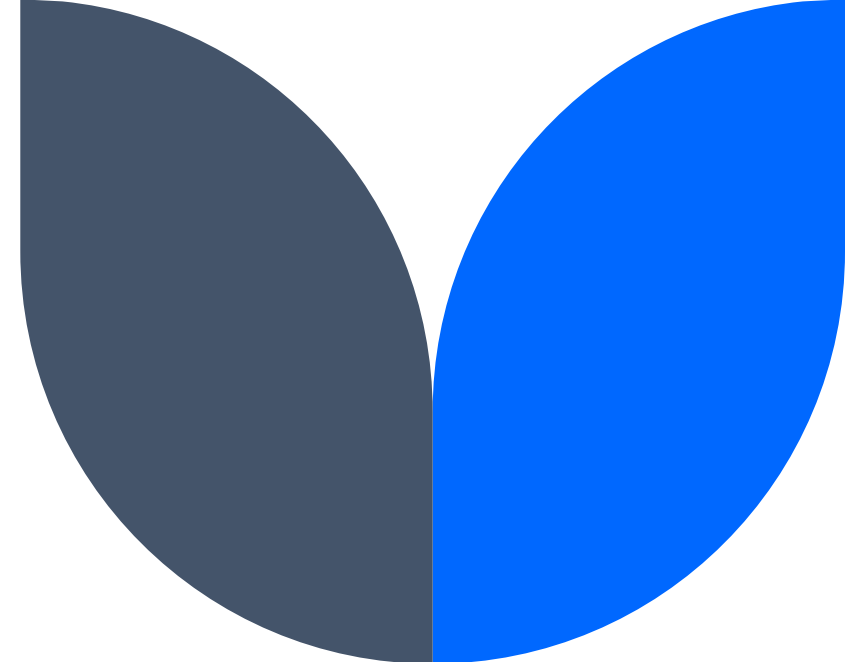
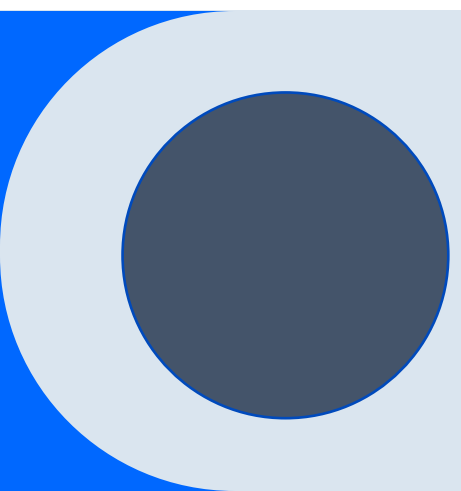




Comparative Performance Analysis of PHP MVC Frameworks

Corey Black - B01651145



BSc (Hons) Web & Mobile Development
Computing Honours Project (COMP10034)

Supervisor: Dr Pablo Salva Garcia
Moderator: Mr Sajjad Bagheri

University of the West of Scotland
February 2026



Research Problem & Contribution

Research Problem

- Limited rigorous, controlled empirical evaluation of PHP framework-level performance.
- Comparative findings are often fragmented and methodologically inconsistent.
- Framework selection is frequently influenced by ecosystem popularity rather than reproducible performance analysis.

Research Gap

- Many comparative studies rely on non-equivalent implementations.
- Runtime and environmental configurations are often inconsistent.
- Micro-benchmarks frequently lack representative CRUD workload realism.
- Controlled studies using functionally equivalent, real-world implementations remain limited.

Research Contribution

- Establishes a controlled, reproducible experimental framework for evaluating framework-level architectural performance.
- Isolated architectural overhead through single independent variable design.
- Provides empirically grounded evidence to support informed, evidence-based framework selection.

Aim and Objectives

Aim

To empirically evaluate and compare the architectural performance characteristics of four major PHP MVC frameworks under controlled, functionally equivalent conditions.

Objectives

1. Implement functionally equivalent CRUD-based applications in Laravel, Symfony, CodeIgniter and Yii.
2. Establish a standardised experimental environment (PHP 8.2.12, Apache, MariaDB) with controlled runtime configuration.
3. Design and execute reproducible benchmarking scenarios using Apache JMeter and Locust.
4. Measure response time (mean, P95), throughput, and memory utilisation under defined concurrency levels.
5. Analyse performance differences in relation to framework architecture (middleware depth, service container overhead, ORM abstraction).

Research Methodology & Experimental Design

Research Approach

- Controlled experimental benchmarking study.
- Single independent variable: PHP MVC framework (Laravel, Symfony, CodeIgniter, Yii).
- Functionally equivalent CRUD applications to isolate architectural overhead.

Controlled Variables

- Identical database schema and fixed dataset size.
- PHP 8.2.12 runtime and consistent Apache/MariaDB configuration.
- Debugging disabled; production-equivalent settings applied.
- Identical workload definitions and concurrency levels.

Dependant Variables (Performance Metrics)

- Response time (mean, P95 tail latency).
- Throughput (requests per second).
- Resource utilisation (CPU and memory consumption).

Benchmarking Framework & Workload Design

Benchmarking Tools

- Apache JMeter – controlled load generation.
- Locust – scripted, behaviour-driven workload simulation.
- Multiple repeated runs per framework for statistical stability.

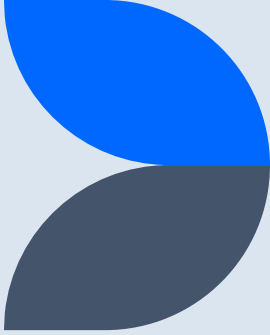
Workload Design

- Standardised CRUD endpoints (Create, Read, Update, Delete).
- Identical request payloads and fixed dataset size.
- No application-level caching enabled.

Load Configuration

- Defined concurrency levels: 1, 10, 50, 100 users.
- Controlled ramp-up periods and warm-up phase.
- Percentile-based analysis (P95) from aggregated results.

Experimental Environment & System Configuration



Hardware & OS

- Windows 11 Home (Version 25H2, Build 26200.7623)
- AMD Ryzen 7 5800X (8-Core Processor)
- 32GB DDR4 RAM
- Localhost benchmarking environment

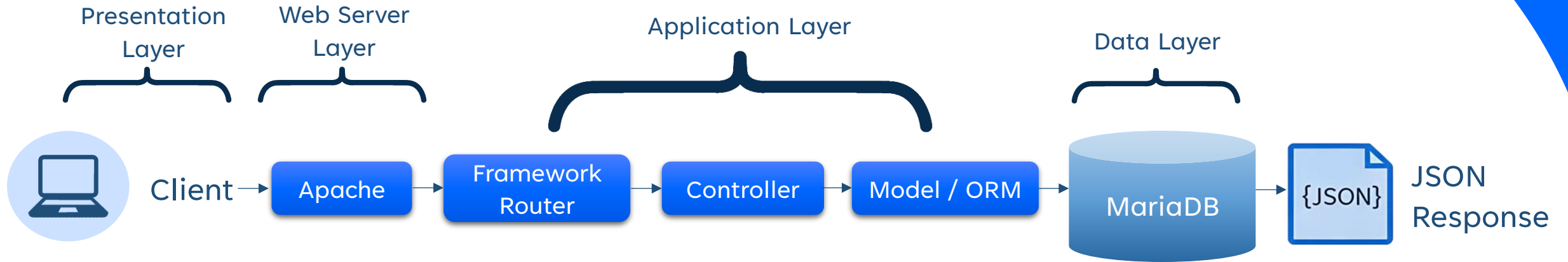
Software Stack

- XAMPP 3.3.0
- Apache 2.4.58 (Win64)
- PHP 8.2.12 (Thread Safe)
- MariaDB 10.4.32 (InnoDB, utf8mb4)
- Composer 2.2.25

Runtime & Configuration Controls

- memory_limit = 128M
- max_execution_time = 30
- OPcache enabled
- display_errors = Off
- Debug mode disabled across all frameworks
- Production-equivalent settings applied
- No framework-specific performance optimisations
- Dependencies frozen for entire benchmarking phase

Standardised Application Architecture



Request Processing Overhead

- Framework-level routing resolution
- Middleware stack traversal
- Service container resolution overhead

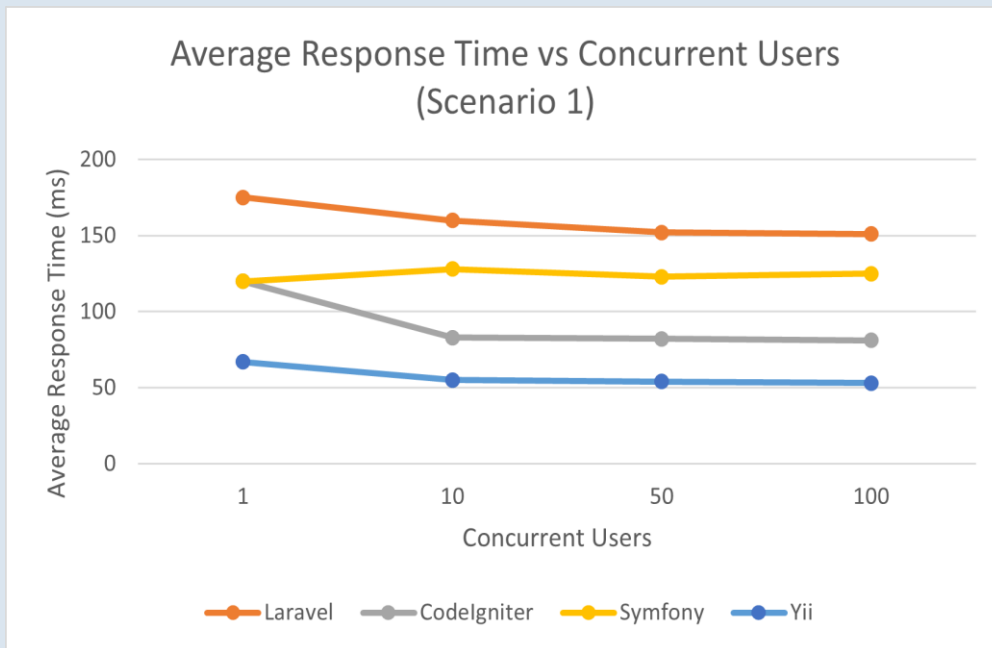
Data Abstraction Overhead

- ORM abstraction layer execution
- Query builder -> SQL generation
- Connection pooling via PDO

Response Serialisation Overhead

- Model hydration / object mapping
- JSON serialisation
- HTTP status + header composition

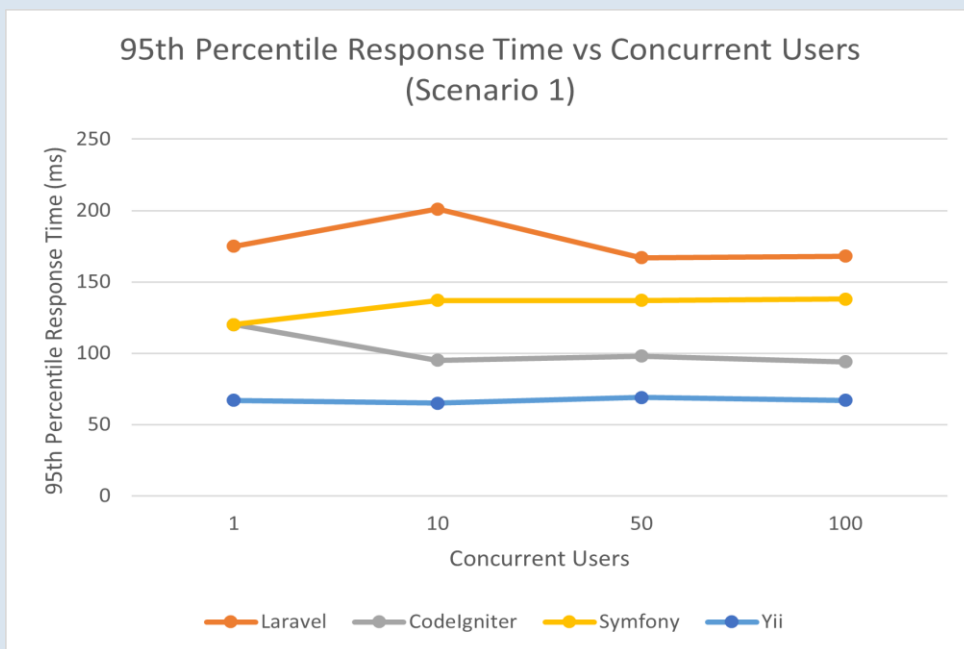
Scenario 1 – Average Response Time Scaling (1-100 Users)



Key Findings

- **Yii achieved the lowest mean response time** decreasing from 67ms (1 user) to 53ms (100 users).
- **CodeIgniter reduced from 120ms to 81ms** under scaling, demonstrating efficient request handling.
- **Laravel decreased from 175ms to 151ms**, but maintained the highest baseline latency.
- **Symfony remained between 120-125ms** under scaling conditions.
- **No non-linear or exponential degradation** observed up to 100 concurrent users.

Scenario 1 – Tail Latency (95th Percentile)



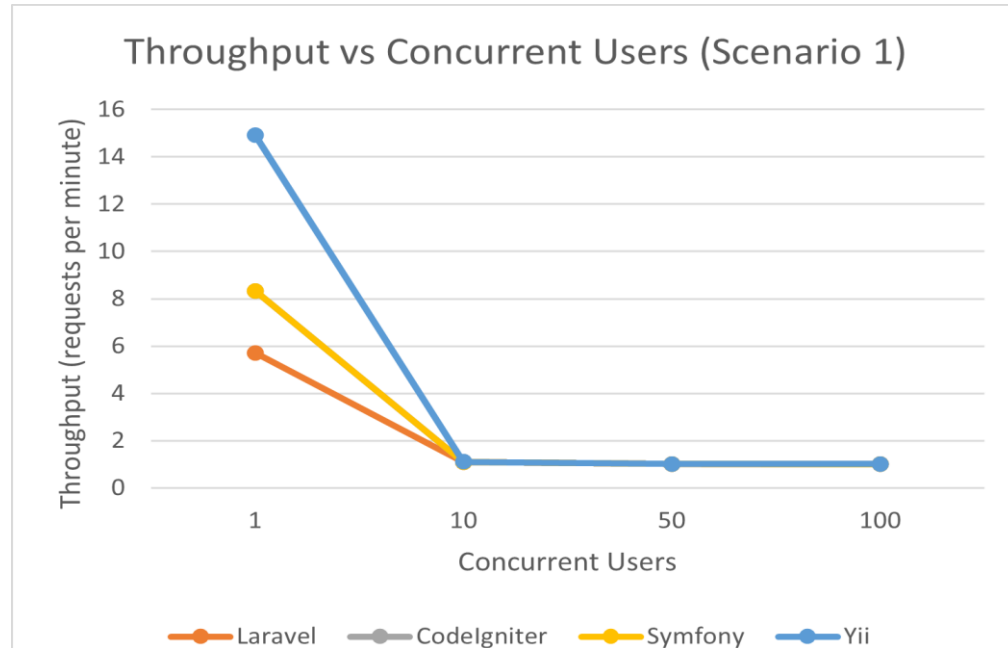
Key Findings

- **Laravel** peaked at 201ms P95 under 10-user concurrency, indicating higher tail variability.
- **Symfony** maintained consistent but elevated P95 values (137-138ms).
- **CodeIgniter** stabilised between 94-98ms P95 under scaling.
- **Yii** maintained the lowest and most stable P95 (65-69ms).
- Stable P95 suggests predictable request pipeline performance under load.

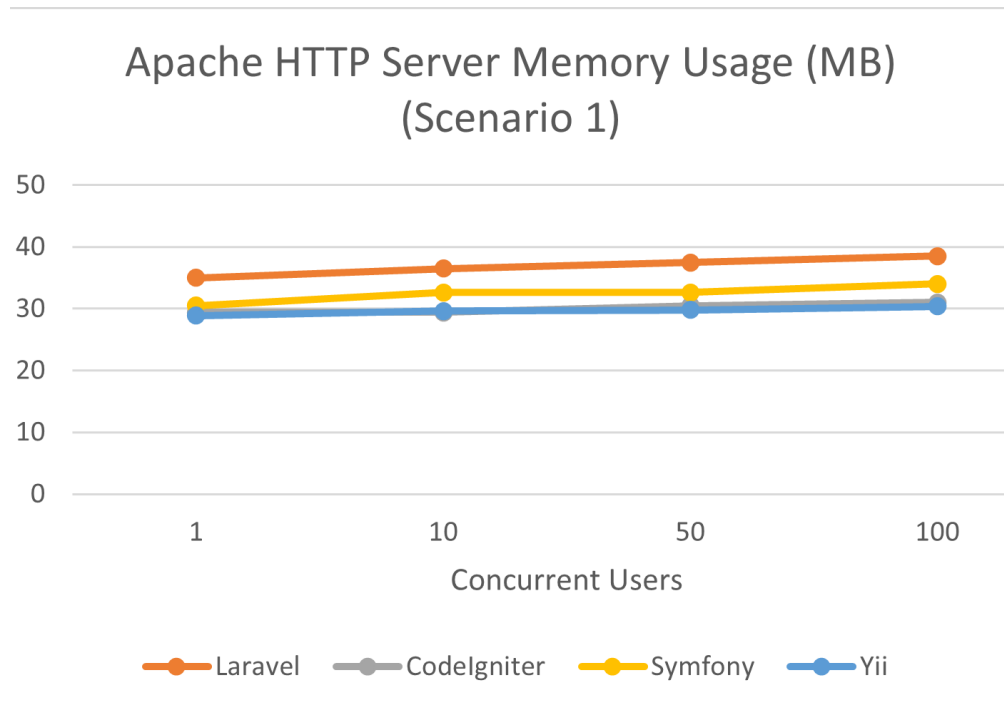
Scenario 1 – Throughput Scaling Behaviour

Key Findings

- At single-user load, **Yii achieved the highest throughput** (14.9 req/min), compared to Laravel (5.7 req/min).
- Under 10+ concurrent users, **throughput converged across frameworks** (~1.0-1.1 req/min).
- **Similar throughput patterns under scaling** suggest **environmental saturation** rather than framework-specific collapse.
- No framework demonstrated exponential throughput degradation.



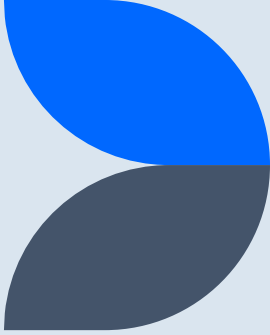
Scenario 1 – Memory Efficiency Comparison



Key Findings

- **Laravel** consumed the highest memory footprint, increasing from ~35MB to ~38MB under scaling.
- **Yii** maintained the lowest memory usage (~29~30MB) across all concurrency levels.
- **CodeIgniter** remained stable around 30-32MB, demonstrating lightweight execution.
- **Symfony** exhibited moderate memory overhead (~31~33MB).
- Memory growth remained linear, with no abnormal allocation spikes.

Scenario 1 – Comparative Performance Summary



Framework	Avg RT @100 Users	P95 RT @10 Users	Memory @ 100 Users	Overall Profile
Laravel	151ms	201ms	~38MB	Feature-rich, higher overhead
Symfony	125ms	137ms	~33MB	Structured, moderate overhead
CodeIgniter	81ms	95ms	~32MB	Lightweight
Yii	53ms	65ms	~30MB	Balanced, lowest overhead

Under controlled read-heavy conditions, **Yii and CodeIgniter demonstrated substantially lower latency and memory overhead than Laravel and Symfony.**

Architectural Factors Influencing Performance

Middleware & Request Pipeline Depth

- Laravel and Symfony implement layered middleware stacks and deeper request pipelines.
- Additional traversal and resolution steps increase baseline latency.
- CodeIgniter and Yii follow a more direct execution path with fewer abstraction layers.

Service Container & Dependency Resolution

- Dynamic dependency injection in Laravel and Symfony introduces runtime resolution overhead.

ORM Abstraction Overhead

- Eloquent (Laravel) and Doctrine-style ORM patterns introduce abstraction cost during query execution.
- CodeIgniter and Yii demonstrated lower overhead in equivalent CRUD operations.

Caching and Lazy Loading Mechanisms

- Yii's lazy loading and internal optimisations likely contributed to improved latency stability.
- Reduced unnecessary object instantiation lowers memory footprint.

Framework Philosophy Trade-Offs

- Feature-rich frameworks prioritise extensibility and maintainability.
- Lightweight frameworks prioritise execution efficiency and lower runtime overhead.

Qualitative Developer Experience Comparison

Implementation Workflow

- Official Composer distributions used for all frameworks.
- Projects structured within unified **/apps** directory inside single Git repository.
- Identical CRUD logic implemented using each framework's native MVC conventions.

Schema & Data Consistency

- Single shared MariaDB schema definition.
- Identical table structure, indexes, and seed dataset applied across all frameworks.
- No framework-specific schema extensions introduced.

Operational Observations

- Setup complexity comparable across frameworks.
- Primary differences emerged at architectural runtime level, not installation stage.
- Variations in developer ergonomics (CLI tooling, configuration verbosity, ORM abstraction depth).

```
E:\Users\corey\Documents\GitHub\Honours-Project\apps\laravel>php artisan --version
Laravel Framework 8.83.29

E:\Users\corey\Documents\GitHub\Honours-Project\apps\laravel>composer show laravel/framework
name      : laravel/framework
descrip.  : The Laravel Framework.
keywords  : framework, laravel
versions  : * v8.83.29
type      : library
license   : MIT License (MIT) (OSI approved) https://spdx.org/licenses/MIT.html#licenseText
homepage  : https://laravel.com
source    : [git] https://github.com/laravel/framework.git d841a226a50c715431952a10260ba4fac9e91cc4
dist      : [zip] https://api.github.com/repos/laravel/framework/zipball/d841a226a50c715431952a10260ba4fac9e91cc4
path      : E:\Users\corey\Documents\GitHub\Honours-Project\apps\laravel\vendor\laravel\framework
names     : laravel/framework, psr/container-implementation, psr/simple-cache-implementation, illuminate/contracts, illuminate/encryption, illuminate/events, illuminate/filesystem, illuminate/hashing, illuminate/http, illuminate/support, illuminate/testing, illuminate/translation, illuminate/validation, illuminate/view
```

Practical Implications for Framework Selection

Performance-Critical or Resource-Constrained Environments

- Yii and CodeIgniter demonstrated lower latency and memory usage under read-heavy workload conditions.
- Suitable for high-traffic APIs or limited-resource hosting environments.

Enterprise-Scale and Modular Systems

- Symfony's structured component architecture supports maintainability and extensibility.
- Appropriate where long-term scalability and modular design outweigh raw performance.

Rapid Development and Ecosystem Support

- Laravel provides extensive tooling, developer ergonomics, and ecosystem maturity.
- Trade-off observed between abstraction richness and runtime efficiency.

Performance vs Maintainability Trade-offs

- Lightweight frameworks prioritise execution efficiency.
- Feature-rich frameworks prioritise architectural flexibility and developer productivity.

Framework selection should align with application requirements, not popularity.

Limitations and Threats to Validity

Controlled Local Testing Environment

- Benchmarks conducted in a non-production local environment.
- Results may differ under distributed or cloud-based deployments.

Single Workload Type (Read-Heavy CRUD)

- Scenario 1 focuses on read-dominant access patterns.
- Write-heavy or compute-intensive workloads may produce different behaviour.

No Advanced Optimisation Techniques Applied

- Default framework configurations used.
- Production-level tuning (caching layers, reverse proxies, OPcache optimisation) not explored.

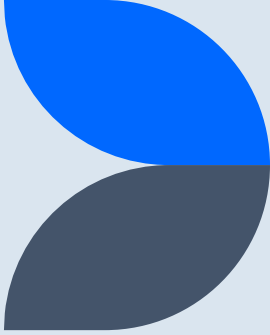
Limited Concurrency Range

- Concurrency evaluated up to 100 users.
- Higher-scale stress testing may reveal different scalability characteristics.

Framework Version and Runtime Dependency

- Results are specific to tested PHP version and framework releases.
- Future updates may alter performance characteristics.

Conclusion and Objective Evaluation



Objective 1 – Develop Functionally Equivalent Applications ✓

- CRUD applications implemented in Laravel, Symfony, CodeIgniter and Yii.
- Identical REST endpoints and database schema.

Objective 2 – Standardised Experimental Environment ✓

- PHP 8.2.12, Apache 2.4.58, MariaDB 10.4.32.
- Debug disabled, OPcache enabled.
- No framework-specific optimisations.

Objective 3 – Reproducible Benchmarking ✓

- Apache JMeter + Locust.
- Controlled concurrency (1-100 users).
- Multiple repeated runs.

Objective 4 – Performance Measurement ✓

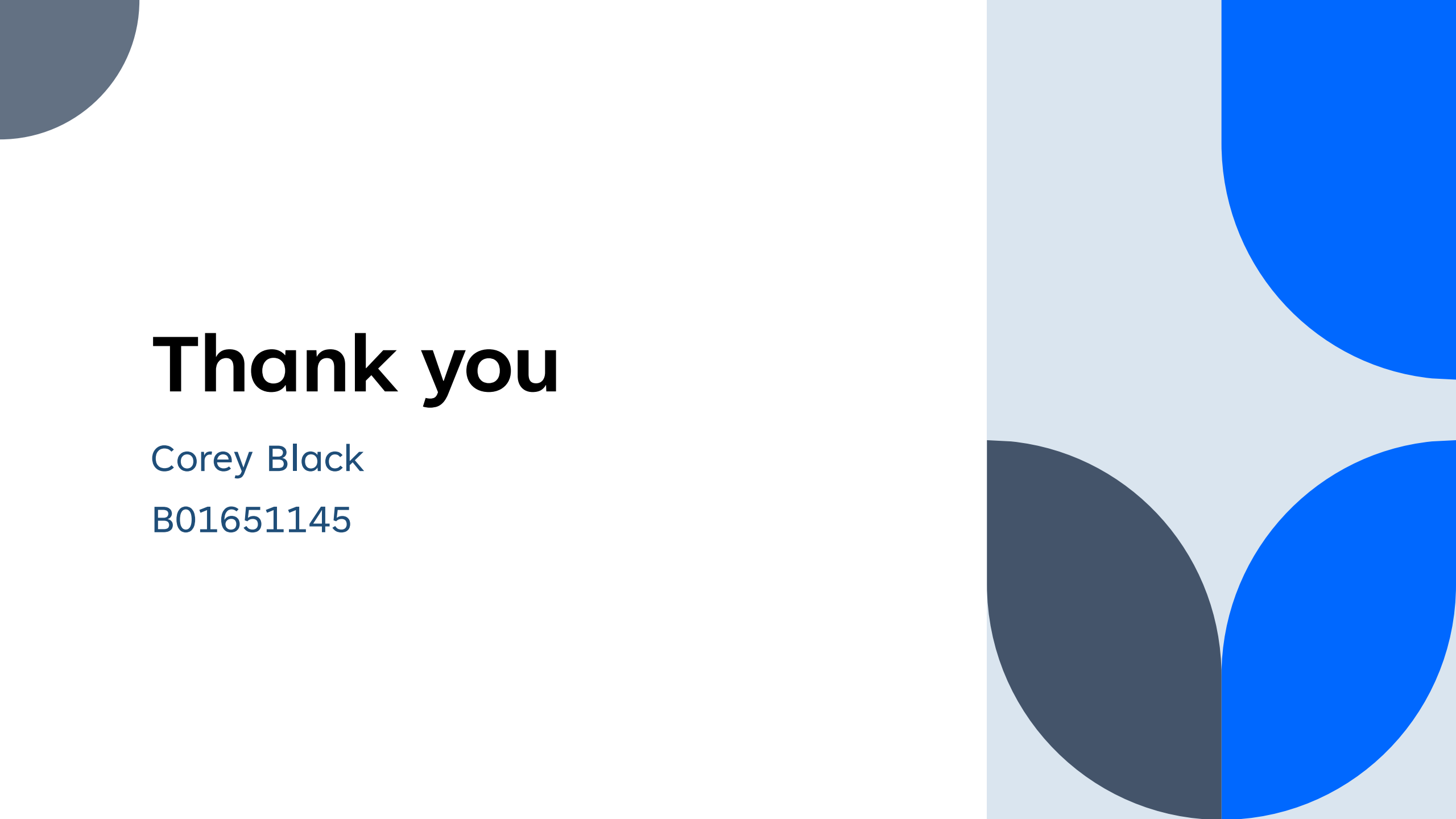
- Mean response time.
- P95 tail latency.
- Throughput scaling.
- Memory usage.

Objective 5 – Architectural Analysis ✓

- Middleware depth examined.
- Service container overhead evaluated.
- ORM abstraction impact analysed.

Current Stand-Point ✓

Under controlled read-heavy conditions, Yii demonstrated the most balanced overall performance profile, followed closely by CodeIgniter.



Thank you

Corey Black

B01651145